# Algorithms for genomic data analysis: Metagenomics data analysis

Aleksander Jankowski

University of Warsaw

05.11.2024

# What is metagenomics?

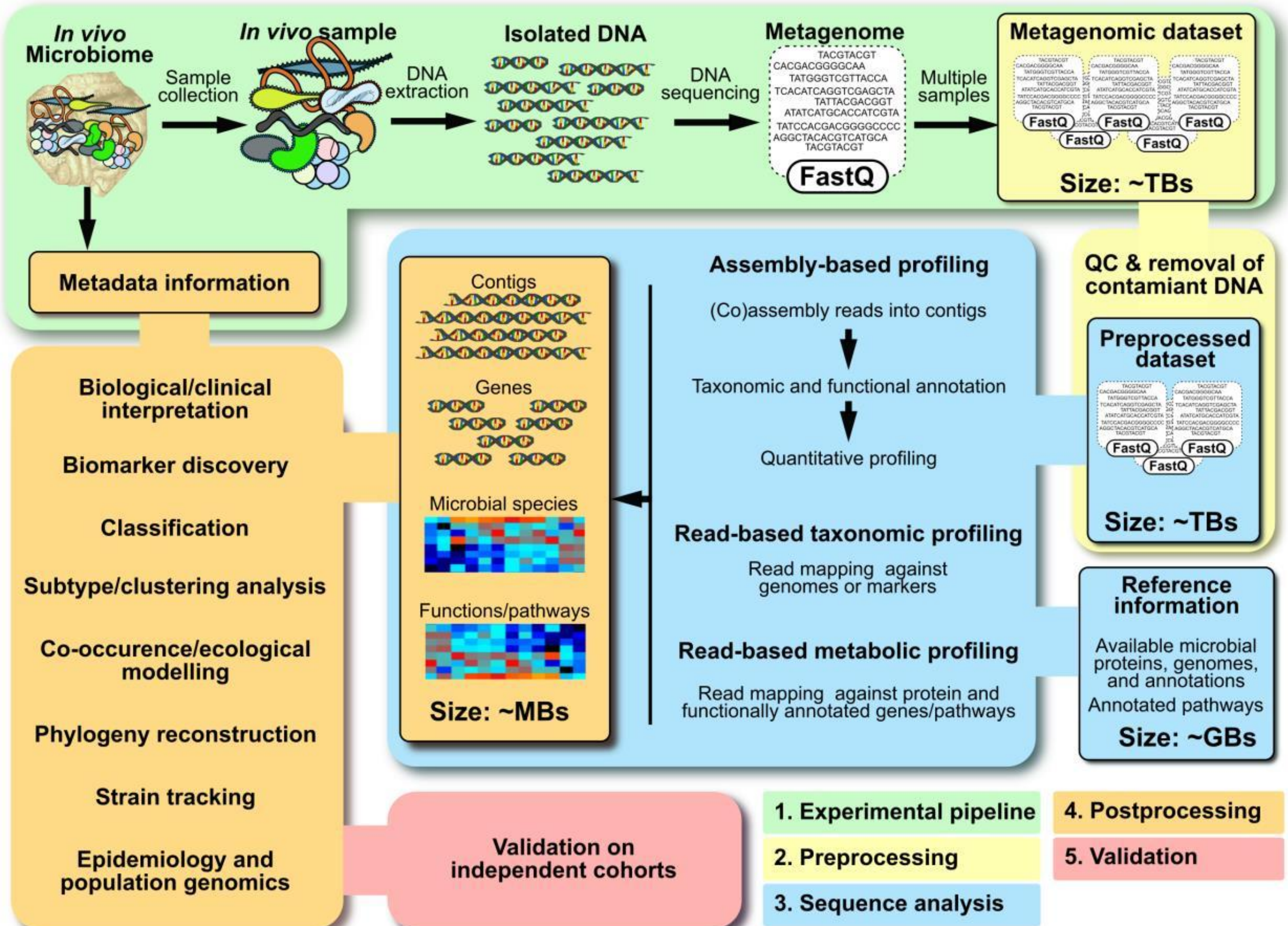
- Metagenomics is the study of the structure and function of nucleotide sequences isolated and analyzed from all the organisms (typically microbes) in a bulk sample.
- Metagenomics is often used to study a specific community of microorganisms, from environments such as:
  - soil,
  - water sample,
  - human skin,
  - human gut.

# Objectives and methods of metagenomic analysis

- The two main objectives in metagenomics data analysis:
  - Who is there? (taxonomic profiling)
  - What can they potentially do? (functional profiling)
- Taxonomic profiling has traditionally been done using marker genes, such as 16S ribosomal RNA gene.
  - Done through amplification to enrich for the given gene via universal primers.
  - Limited resolution because of conservation of the marker genes.
- Contemporary approach: shotgun metagenomics (sequencing the whole DNA material from a community):
  - enables the discovery of unknown taxa
  - allows to predicting the functions of microbial community members.

# Overview of shotgun metagenomics

Quince *et al.*,
Nat Biotechnol 2017



# Concept: marker genes

- In metagenomics, *marker genes* are conserved genes containing one or more highly variable regions, which allow to discriminate between different taxonomic lineages.
- Ribosomal RNA genes (in particular 16S rRNA for bacteria and 18S rRNA for eukaryotes) are commonly used as marker genes to study phylogenetic relationships.
- 16S ribosomal RNA gene:
  - encodes the key component of the small subunit of a prokaryotic ribosome,
  - is found in most bacteria and archaea, usually in many copies,
  - contains 9 hypervariable regions separated by highly conserved sequences.

# Concept: operational taxonomic units (OTUs)

- Operational taxonomic units (OTUs) are groups of similar genomes (usually ≥97% identity) considered to be roughly approximate to ‘species’.
- When the genomes are not known, often dened as a metagenome sequencing cluster.
    - If such is the case, the number of estimated OTUs can be higher than the real number of species.
    - The actual taxonomic composition and hierarchy of OTUs depends on cluster homogeneity and reference database used (if any).

**Diversity measures:**

- *Alpha diversity:* the diversity measured within a particular ecosystem or sample. Commonly characterized by OTU richness, evenness and phylogenetic diversity.
- *Beta diversity:* diversity comparison between particular ecosystems or samples. Commonly analysed through ecological and phylogenetic distances estimated from the sample composition.

# Binning approaches in metagenomics

- Binning is the process of grouping reads into individual genomes and assigning the groups to specific taxonomic unit (subspecies < species < genus < family < order < …).
- Two approaches to binning:
  - *Composition-based binning* – based on the observation that individual genomes have a unique distribution of *k*-mer sequences (*genomic signature*). By making use of this species-specific composition, sequences can be grouped into respective genomes.
  - *Similarity- or homology-based binning* – uses alignment algorithms (e.g. BLAST), hidden Markov Models (HMMs) or other methods to quantify similarity against available genome databases (e.g. NCBI RefSeq). Sequences are then binned according to their assigned taxonomic information.

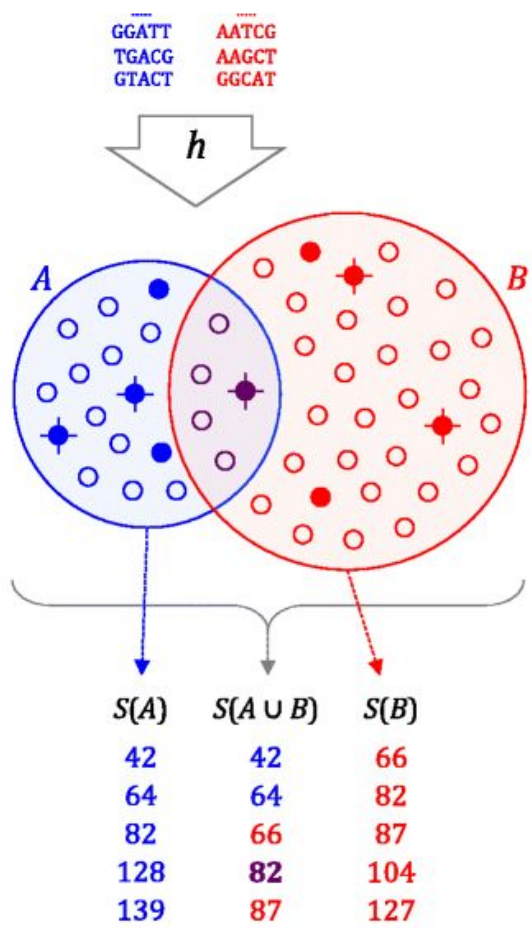
# Recap: minhashes

- Consider a common problem: how much overlap do two sets have?
- Common measure: Jaccard index $J(A,B) = \frac{|A \cap B|}{|A \cup B|}$
- Typical application: A, B – sets of *k*-mers found in two metagenomics samples
- Explicitly building and intersecting sets of *k*-mers yields unacceptable computing times.
- The trick: instead of using the set of all *k*-mers from each sample, just pick some representation:
    - One representative *k*-mer from each sample? Lexicographically smallest (minimizer)?
    - More unbiased approach instead of taking the minimal *k*-mer, apply a hash function, and take the *k*-mer with minimal hash value.
    - Note a solution that allows for a more nuanced estimate: take a number of hash functions (multiple saltings).
    - MinHash (Broder 1997) originally applied to document comparison.

# Recap: sketch representations (sketches)

- Reduce large sets (e.g. sets of *k*-mers) to compressed sketch representations.
- Similarity of the original sets can be rapidly estimated, hopefully with some bounded error.
- Importantly, a desirable sketch would have computation error dependent only on the sketch size (and independent of the size of compared sets — i.e. independent of the genome size).

# MinHash bottom sketch strategy in Mash




Ondov *et al.*, Genome Biol 2016

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{|S(A \cup B) \cap S(A) \cap S(B)|}{|S(A \cup B)|}$$

# MinHash bottom sketch strategy in Mash

- Mash (Ondov *et al.*, Genome Biol 2016) provides two basic functions: ***sketch*** and ***dist***.
- The ***sketch*** function converts a sequence or a collection thereof into a MinHash sketch.
  - Note: the inputs can be nucleotide sequences, whole genomes, metagenomes, or just raw sequencing reads.
- The ***dist*** function compares two sketches and returns:
  - an estimate of the Jaccard index (i.e. the fraction of shared bottom sketch *k*-mers),
  - a *p*-value,
  - the Mash distance, which makes a simplistic estimate of the rate of sequence mutation.

# MinHash bottom sketch strategy in Mash in detail

- For nucleotide sequences, Mash uses canonical *k*-mers to allow strand-neutral comparisons.
    - Only the lexicographically smaller of the forward and reverse complement *k*-mer is hashed. (Works well if $k$ is odd.)
- For a given sketch size $s$, Mash returns the $s$ smallest hashes returned by the chosen hash function over all canonical *k*-mers. (These are referred to as “bottom sketches”.)
- For a sketch size $s$ and genome size $n$, a bottom sketch can be efficiently computed in $O(n \log s)$ time.
    - Maintain a sorted list of size $s$ and update the current sketch only when a new hash is smaller than the current sketch maximum.
    - The probability that the $i$-th hash of the genome will enter the sketch is $s / i$, and the time needed to handle such a case is $O(\log s)$, so the expected run time of the algorithm is $O(n + s \log s \log n)$.

# Choice of *k*-mer size for MinHash

- The probability of a given *k*-mer *K* appearing in a random genome *X* of size *n* is:

$$P(K \in X) = 1 - \left(1 - |\Sigma|^{-k}\right)^{n}$$

  where $\Sigma = \{A,C,G,T\}$.

- For $k = 16$ the probability of observing a given k-mer in a 3 Gbp genome is 0.50 and 25% of k-mers are expected to be shared between two random 3 Gbp genomes by chance alone.

- To avoid this phenomenon, it is sufficient to choose a value of *k* that minimizes the probability of observing a random *k*-mer.

- Given a known genome size *n* and the desired probability *q* of observing a random *k*-mer (e.g. 0.01), the resulting minimal *k* can be computed as

$$k' = \left\lceil \log_{|\Sigma|} \left(n\left(1 - q\right) / q\right) \right\rceil$$

  which yields $k = 14$ and $k = 19$ for 5 Mbp and 3 Gbp genomes ($q = 0.01$), respectively.

- Commonly used for metagenomes: $k = 21$ and $s = 1000$

- When memory is the limit: $k = 16$ was chosen so that each hash could fit in 32 bits.

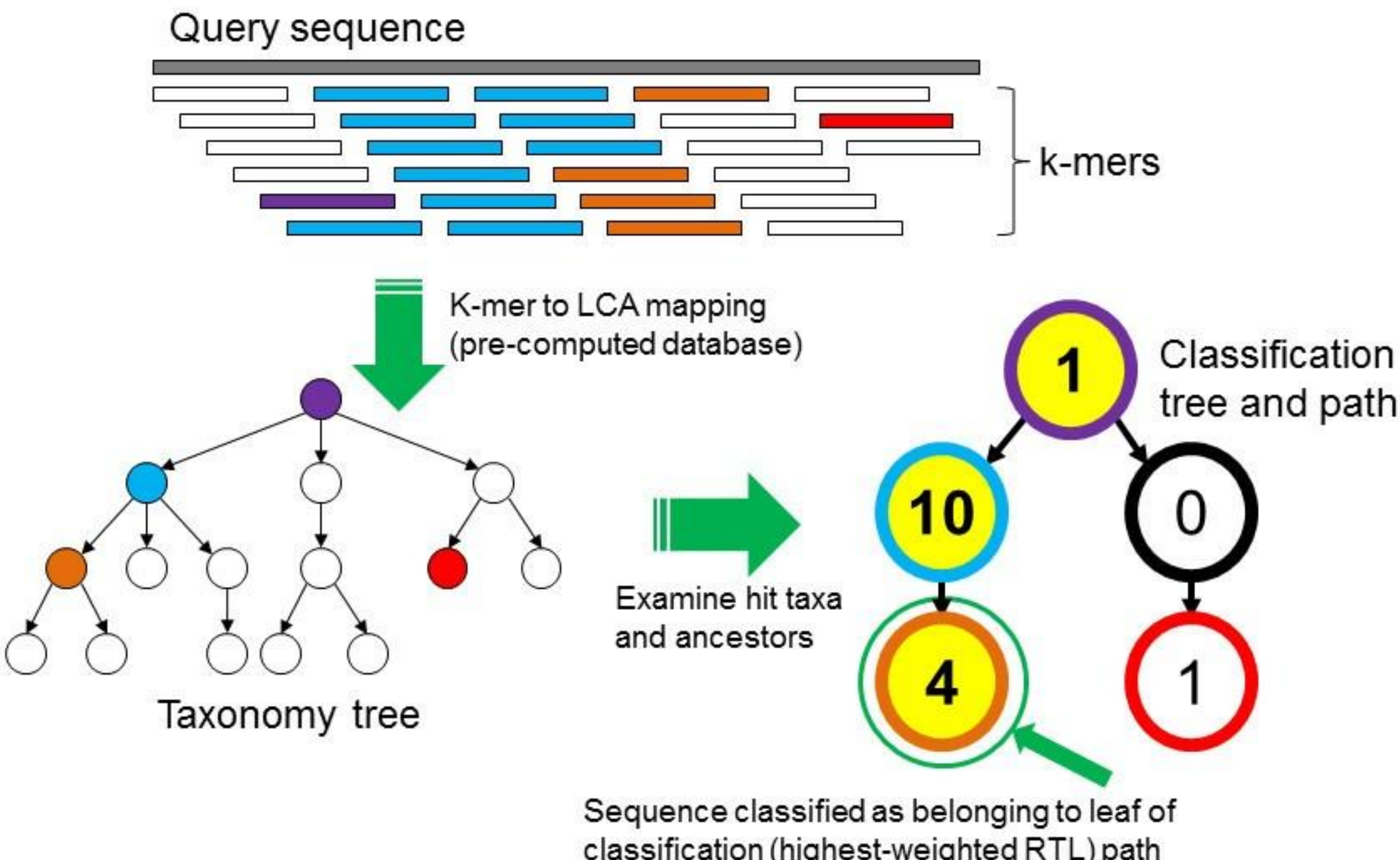
# A related sketch strategy: FracMinHash

- Consider a hash function in the domain $[0, H]$.
- Instead of keeping the $s$ smallest hashes, we keep all hashes $h$ such that $h \leq H / s$.
  - The fraction $H / s$ is the *maximum hash value* allowed in the FracMinHash sketch.
- While FracMinHash keeps the selection of the smallest elements from MinHash, its size is dynamic.
  - This allows for a more reliable estimate of Jaccard index and containment index.
- FracMinHash has been proposed in (Irber *et al.*, bioRxiv 2022).

# A simplistic homology-based binning approach

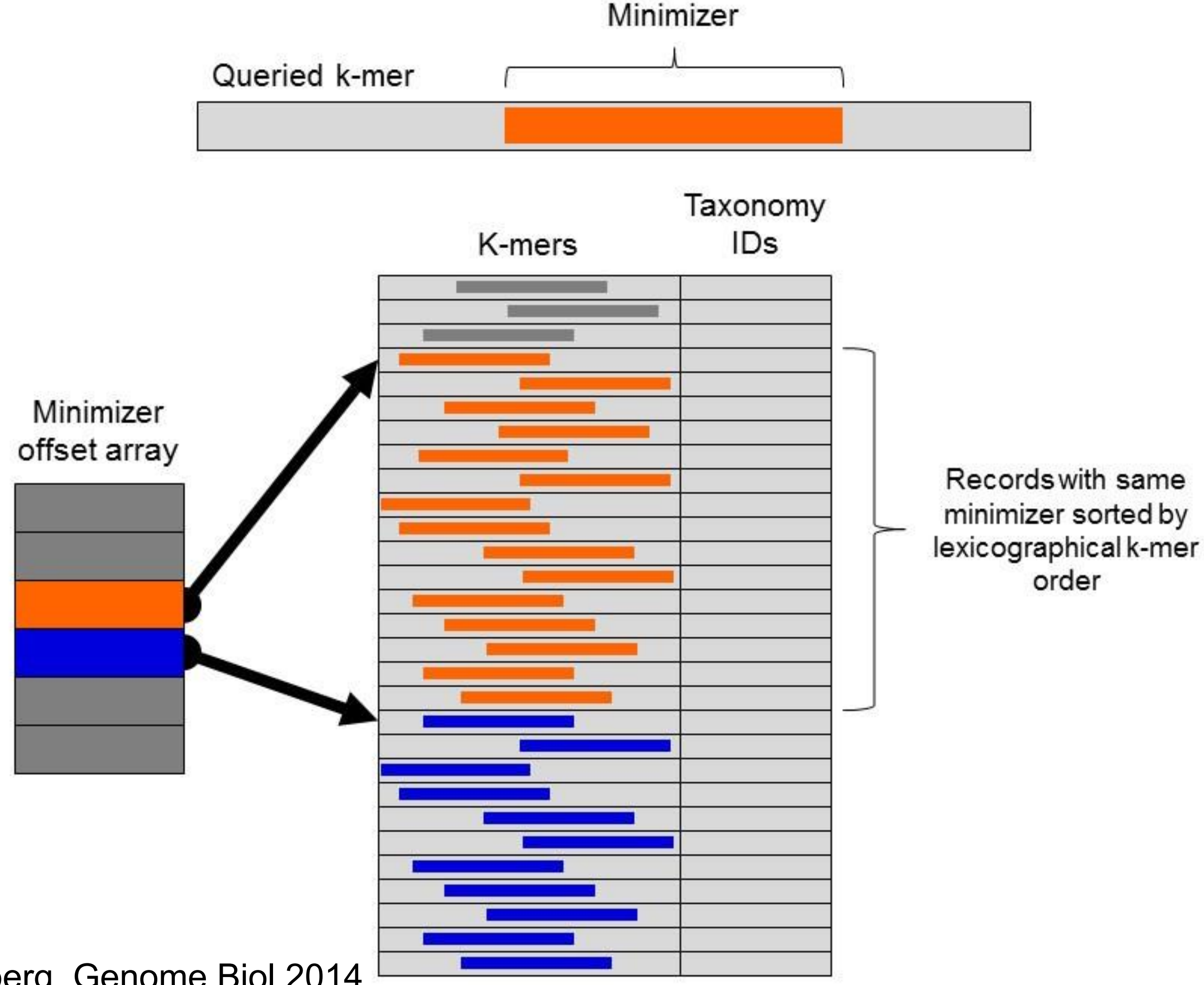
- Prepare a database consisting of:
    - a set of reference sequences (either marker genes only or whole genomes),
    - a taxonomic tree of corresponding organisms (subtree of the *tree of life*).
- For each sequencing read:
    - BLAST it against the set of reference sequences,
    - find within the tree a lowest common ancestor (LCA) of organisms having BLAST hits.

# Homology-based binning in Kraken




Wood and Salzberg, Genome Biol 2014

# Kraken database translating *k*-mers to LCAs



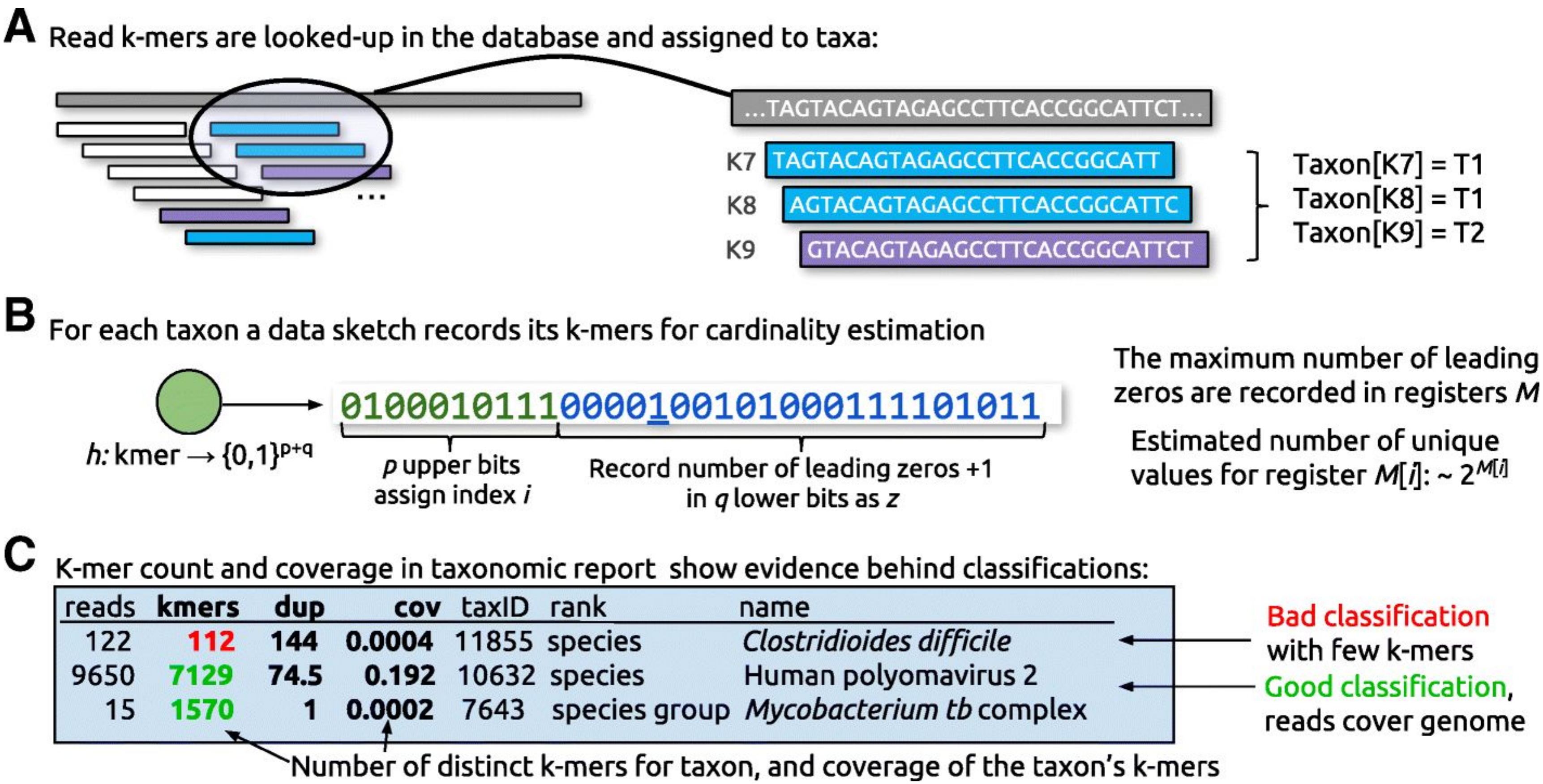

default values:
k = 31, l = 15

Wood and Salzberg, Genome Biol 2014

# Variants of Kraken (version 1)

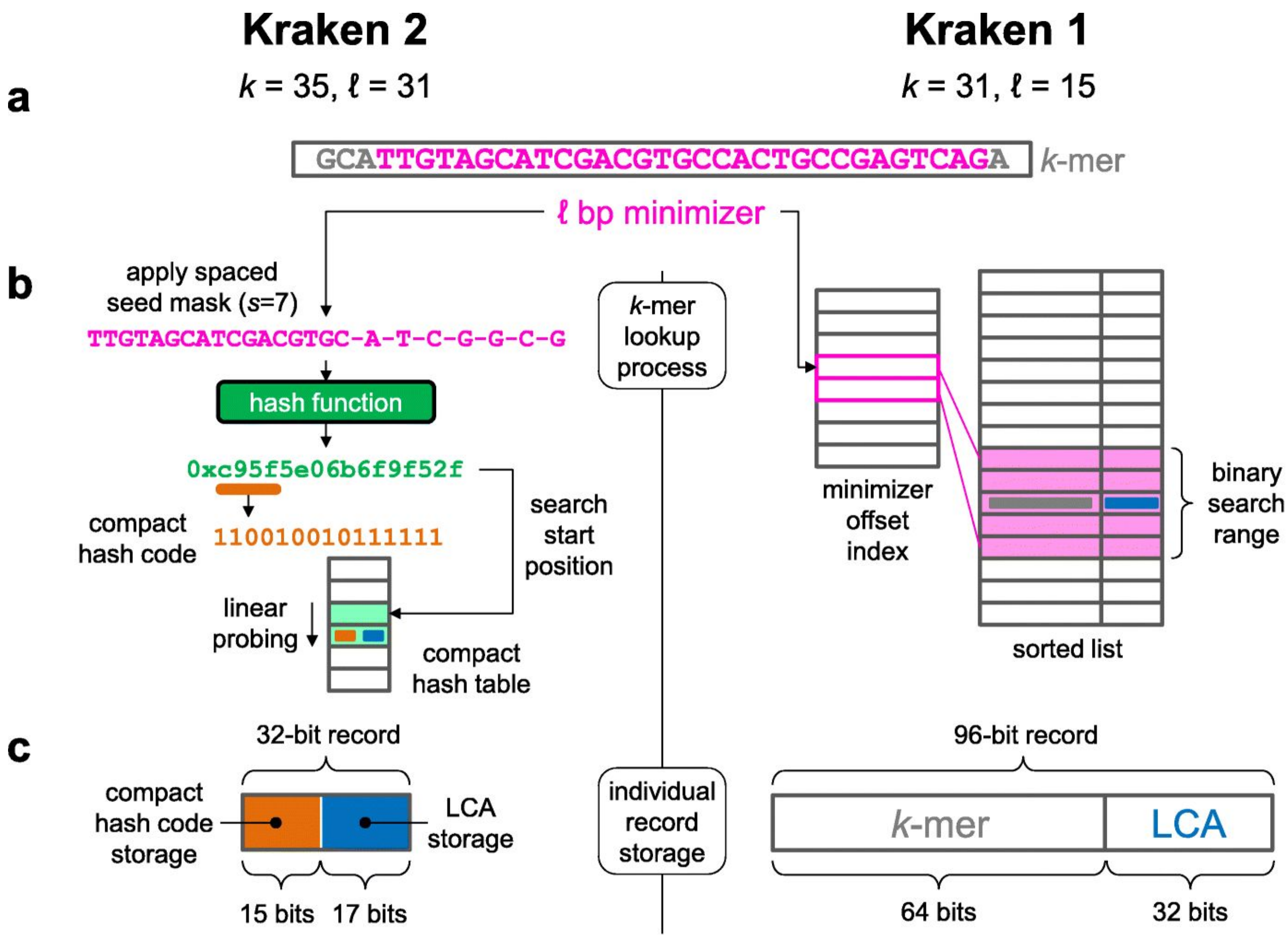
- Kraken: requires 70 GB memory (a major bottleneck in 2014!) for a database of 2,256 complete NCBI RefSeq genomes.
- MiniKraken: removed 18/19 of the database records (i.e. 18/19 *k*-mers).
- Kraken-Q and MiniKraken-Q: ‘quick operation’ mode, where instead of querying all the *k*-mers in a sequence against the database, the operation is stopped at the first *k*-mer that exists in the database.
- Kraken-GB: extended database that also contains all GenBank’s draft and completed genomes for bacteria and archaea (8,517 genomes in total).

# KrakenUniq: similar to the original, but also determines *k*-mer coverage for each taxonomic classification




Breitwieser *et al.*, Genome Biol 2018

# Kraken 2: reduced memory usage and run time




Wood *et al.*, Genome Biol 2019

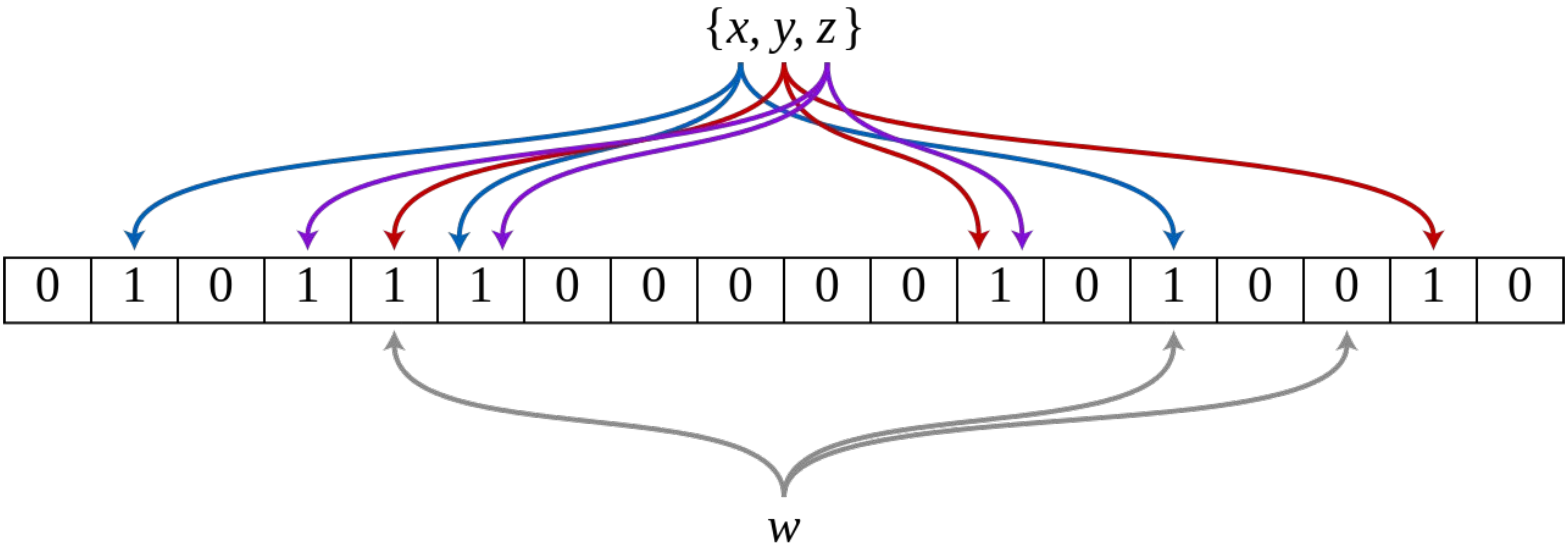
# Summary of a selection of tools for taxonomic classification using homology-based binning

- Kraken (2014): taxonomic classification using exact k-mer matching. Used minimizers to improve time efficiency.
- KrakenUniq (2018), based on Kraken 1, using the same databases and classification algorithm. KrakenUniq also determines the k-mer coverage for each taxonomic classification. This metric is valuable in filtering out false positive reads.
- Kraken 2 (2018): significant memory and speed improvements compared to Kraken 1, at a price of tiny (~1%) false-positive rate (erroneously assigning a read to a species) that KrakenUniq does not have.
    - For diagnosis of infections, where the goal is to identify a very small number of reads from non-host genome, use KrakenUniq.
- Bracken (2017): not a taxonomic classification program, but a a post-processing script for the tools above. It provides abundance estimation at any desired taxonomic level (e.g. species or genus). This allows to see the estimated composition of the sample at a particular taxonomic level.

# How to remove erroneous *k*-mers?

- True k-mers will appear multiple times in the input, while false k-mers will appear only a few times, possibly once only.
- Given a coverage threshold c, we can ignore such low-abundance k-mers with counts less than c as follows (two-stage MinHash filter strategy):
    - Track both the k-mer hashes in the current sketch and a secondary set of candidate hashes.
    - At any time, the current sketch contains the s smallest hashes of all k-mers that have been observed at least c times and the candidate set contains hashes that are smaller than the largest value in the sketch (sketch max), but have been observed less than c times.
    - When processing new k-mers, if a new hash is smaller than the current sketch max, it is checked against the candidate set, and possibly removed from the candidate set and added to the sketch.
    - The result of this online method is equivalent to running the MinHash algorithm on only those k-mers that occur c or more times in the input. However, the size of the candidate set is only limited by the number of all k-mers.
- Alternatively, a Bloom filter can be used to probabilistically exclude single-copy k-mers using a fixed amount of memory.

# Bloom filter to probabilistically exclude single-copy *k*-mers




https://doris.apache.org/

# Jaccard index and p-value estimation

- To estimate the Jaccard index, we merge-sort two bottom sketches S(A) and S(B).
- The merge is terminated after s unique hashes have been processed (or both sketches exhausted). This requires only O(*s*) time, because the sketches are stored in sorted order, and effectively calculates

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{|S(A \cup B) \cap S(A) \cap S(B)|}{|S(A \cup B)|}$$

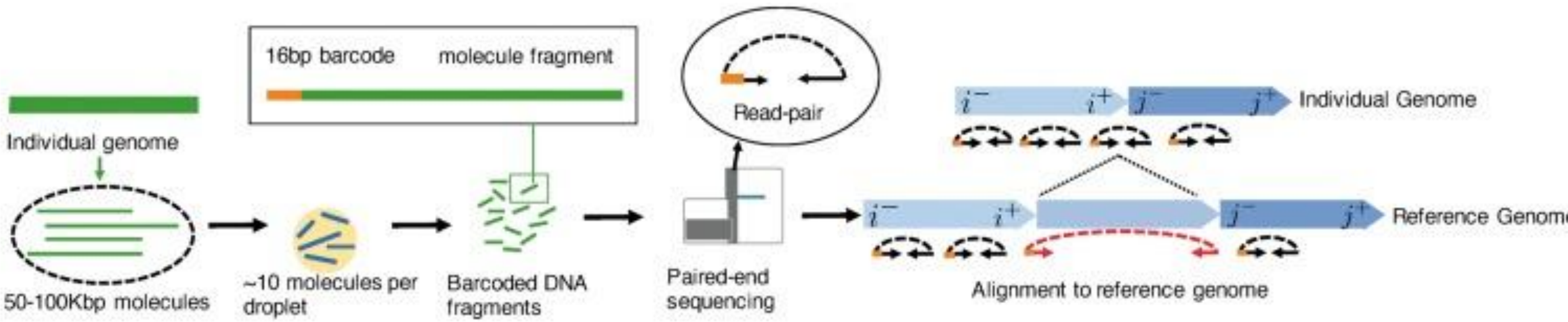
- It can be shown that the standard error of the Jaccard estimate relies only on the sketch size and is independent of genome size

$$\varepsilon = O\left(\frac{1}{\sqrt{s}}\right)$$

- The relative error can grow quite large for very small Jaccard values (i.e. divergent genomes). In these cases, a larger sketch size or smaller *k* is needed to compensate.
- The p-value is calculated from the binomial distribution with probability *w* / *m*, where *m* — number of all distinct k-mers in A and B, *w* — number of shared k-mers, and number of trials *s*.

# Deconvolution of linked reads

- Linked reads carry information about longer stretches of sequence through a 3'-end barcode.
    - Example technology: 10x Genomics system

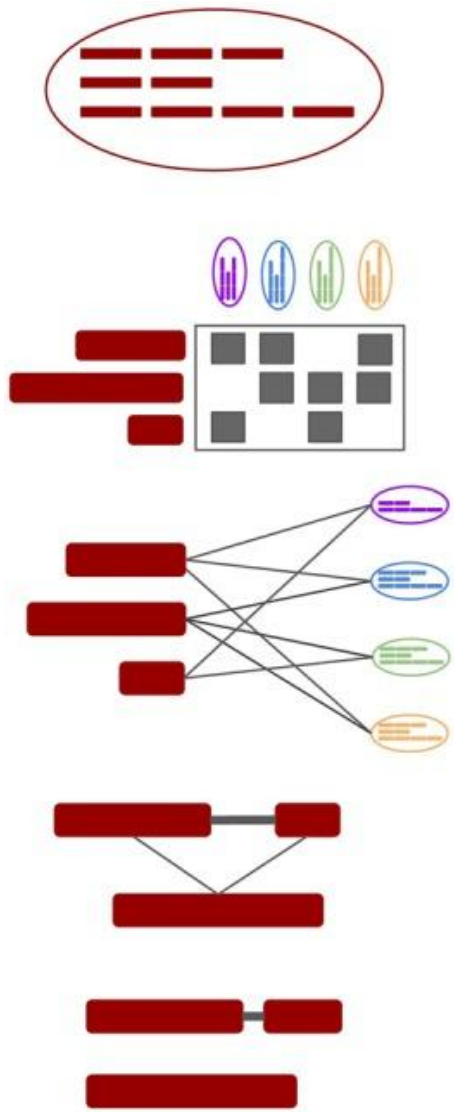



Elyanow *et al.*, Bioinformatics 2018

# Deconvolution of linked reads

- Linked reads carry information about longer stretches of sequence through a 3’-end barcode.
  - Example technology: 10x Genomics system
- Each group of reads that share a 3’ barcode has an unobserved set of longer fragments from which each read was obtained.
- Barcode deconvolution problem: assign each read with a given 3’ barcode to a group corresponding to the longer fragments from which the reads were sampled.

# Deconvolution of linked reads in Minerva



We obtain reads with the same barcode grouped into a read cloud

For each read cloud reads are mapped to other read clouds

A bipartite graph is constructed between reads and read clouds

A graph between reads is constructed

Reads are clustered into groups